

Subobjects copy elision

I. Quick Introduction

[class.copy.elision] in the current working draft [N4713] provides a whitelist of cases when copy elision is permitted. With the current strict rules, it is impossible to elide copying while returning a subobject, so users of the C++ language have to write code like `return std::move(some_pair.second);` in order to get better performance. But even with `std::move` the generated code may be not as good as it could be with the copy elided.

This paper proposes to relax the [class.copy.elision] rules in order to allow compilers to produce better code.

II. Motivating example #1

Consider the following code:

```
#include <utility>
#include <string>

std::pair<std::string, std::string> produce();

static std::string first_non_empty() {
    auto v = produce();
    if (!v.first.empty()) {
        return v.first;
    }
    return v.second;
}

int example_1() {
    return first_non_empty().size();
}
```

Currently, compilers compile `example_1()` to the following steps:

1. call `produce()` to initialize `v`
2. evaluate the condition of the `if` statement
3. copy construct a `std::string` from either `v.first` or `v.second`
4. call the destructor for `v`
5. return the size of the copy constructed `std::string`

User may try to optimize the `first_non_empty()` by adding `std::move` to turn the copy construction of the string into a move construction.

```
#include <utility>
#include <string>

std::pair<std::string, std::string> produce();

static std::string first_non_empty_move() {
    auto v = produce();
    if (!v.first.empty()) {
        return std::move(v.first);
    }
    return std::move(v.second);
}

int example_1_move() {
    return first_non_empty_move().size();
}
```

Unfortunately, the above improvement **requires a highly experienced user** and still **does not produce an optimal result**: compilers are still not always able to optimize out all the unnecessary operations, even after inlining.

Ideally, compilers should be allowed to optimize away subobjects' copying. In that case the compiler would be able to optimize the code to the following:

```
#include <utility>
#include <string>

std::pair<std::string, std::string> produce();

int example_1_optimized() {
    auto v = produce();
    if (!v.first.empty()) {
        return v.first.size();
    }
    return v.second.size();
}
```

Compilers could do the above optimization if the callee is inlined and subobject copy elision were allowed (as proposed in this paper). In that case, a compiler could allocate enough stack space for storing the owning object *v* in the caller, call the destructors of non-returned-from-the-callee subobjects and proceed with the subobject as a return value.

But [class.copy.elision] currently prevents that optimization:

This elision of copy/move operations, called copy elision, is permitted in the following circumstances (which may be combined to eliminate multiple copies):

- in a return statement in a function with a class return type, when the expression is the name of a **non-volatile automatic object** (other than a function parameter or a variable introduced by the exception-declaration of a handler (18.3)) with the same type (ignoring cv-qualification) as the function return type, the copy/move operation can be omitted by constructing the automatic object directly into the function call's return object

Here are the compiler-optimized assemblies for the above 3 code snippets:

example_1() https://godbolt.org/g/V6KqWv	example_1_move() https://godbolt.org/g/3xZvtw	example_1_optimized() https://godbolt.org/g/RiTmPE
<pre> .LC0: .string "basic_string::_M_construct null not valid" .LC1: .string "basic_string::_M_create" <skip>::_M_construct(<skip>) push rbp push rbx mov rbp, rdi sub rsp, 24 test rsi, rsi jne .L2 test rdx, rdx jne .L19 .L2: mov rbx, rdx sub rbx, rsi cmp rbx, 15 ja .L20 mov rdx, QWORD PTR [rbp+0] cmp rbx, 1 mov rax, rdx je .L21 test rbx, rbx jne .L5 mov QWORD PTR [rbp+8], rbx mov BYTE PTR [rdx+rbx], 0 add rsp, 24 pop rbx pop rbp ret .L21: movzx eax, BYTE PTR [rsi] mov BYTE PTR [rdx], al mov rdx, QWORD PTR [rbp+0] mov QWORD PTR [rbp+8], rbx mov BYTE PTR [rdx+rbx], 0 add rsp, 24 pop rbx pop rbp ret .L20: test rbx, rbx js .L22 lea rdi, [rbx+1] mov QWORD PTR [rsp+8], rsi call operator new(unsigned long) mov rsi, QWORD PTR [rsp+8] mov QWORD PTR [rbp+0], rax mov QWORD PTR [rbp+16], rbx .L5: mov rdx, rbx mov rdi, rax call memcpy mov rdx, QWORD PTR [rbp+0] mov QWORD PTR [rbp+8], rbx mov BYTE PTR [rdx+rbx], 0 add rsp, 24 pop rbx pop rbp ret .L19: mov edi, OFFSET FLAT:.LC0 call std::__throw_logic_error .L22: mov edi, OFFSET FLAT:.LC1 call std::__throw_length_error example_1(): push rbp </pre>	<pre> example_1_move(): push rbp push rbx sub rsp, 104 lea rdi, [rsp+32] mov rbx, rsp call produce[abi:cxx11]() mov rax, QWORD PTR [rsp+40] test rax, rax je .L2 lea rdx, [rbx+16] lea rcx, [rsp+48] mov QWORD PTR [rsp], rdx mov rdx, QWORD PTR [rsp+32] cmp rdx, rcx je .L13 mov QWORD PTR [rsp], rdx mov rdx, QWORD PTR [rsp+48] mov QWORD PTR [rsp+16], rdx .L4: mov QWORD PTR [rsp+8], rax lea rax, [rsp+48] mov rdi, QWORD PTR [rsp+64] mov QWORD PTR [rsp+40], 0 mov BYTE PTR [rsp+48], 0 mov QWORD PTR [rsp+32], rax lea rax, [rsp+80] cmp rdi, rax je .L6 call operator delete(void*) jmp .L6 .L2: lea rax, [rbx+16] lea rdx, [rsp+80] mov QWORD PTR [rsp], rax mov rax, QWORD PTR [rsp+64] cmp rax, rdx je .L14 mov QWORD PTR [rsp], rax mov rax, QWORD PTR [rsp+80] mov QWORD PTR [rsp+16], rax .L8: mov rax, QWORD PTR [rsp+72] mov QWORD PTR [rsp+8], rax .L6: mov rdi, QWORD PTR [rsp+32] lea rax, [rsp+48] cmp rdi, rax je .L9 call operator delete(void*) .L9: mov rdi, QWORD PTR [rsp] add rbx, 16 mov rbp, QWORD PTR [rsp+8] cmp rdi, rbx je .L1 call operator delete(void*) .L1: add rsp, 104 mov eax, ebp pop rbx pop rbp ret .L14: movdqa xmm0, XMMWORD PTR [rsp+80] movaps XMMWORD PTR [rsp+16], xmm0 jmp .L8 .L13: movdqa xmm0, XMMWORD PTR [rsp+48] </pre>	<pre> example_1_optimized(): push rbx sub rsp, 64 mov rdi, rsp call produce[abi:cxx11]() mov rax, QWORD PTR [rsp+8] test rax, rax mov ebx, eax jne .L3 mov ebx, DWORD PTR [rsp+40] .L3: mov rdi, QWORD PTR [rsp+32] lea rax, [rsp+48] cmp rdi, rax je .L4 call operator delete(void*) .L4: mov rdi, QWORD PTR [rsp] lea rdx, [rsp+16] cmp rdi, rdx je .L1 call operator delete(void*) .L1: add rsp, 64 mov eax, ebx pop rbx ret </pre>

<pre> push rbx sub rsp, 104 lea rdi, [rsp+32] mov rbx, rsp call produce[abi:cxx11]() mov rdx, QWORD PTR [rsp+40] lea rax, [rbx+16] mov QWORD PTR [rsp], rax test rdx, rdx je .L24 mov rsi, QWORD PTR [rsp+32] mov rdi, rbx add rdx, rsi call <skip>::_M_construct(<skip>) .L27: mov rdi, QWORD PTR [rsp+64] lea rax, [rsp+80] cmp rdi, rax je .L26 call operator delete(void*) .L26: mov rdi, QWORD PTR [rsp+32] lea rax, [rsp+48] cmp rdi, rax je .L28 call operator delete(void*) .L28: mov rdi, QWORD PTR [rsp] add rbx, 16 mov rbp, QWORD PTR [rsp+8] cmp rdi, rbx je .L23 call operator delete(void*) .L23: add rsp, 104 mov eax, ebp pop rbx pop rbp ret .L24: mov rsi, QWORD PTR [rsp+64] mov rdx, QWORD PTR [rsp+72] mov rdi, rbx add rdx, rsi call <skip>::_M_construct(<skip>) .L27 mov rdi, QWORD PTR [rsp+64] mov rbx, rax lea rax, [rsp+80] cmp rdi, rax je .L32 call operator delete(void*) .L32: mov rdi, QWORD PTR [rsp+32] lea rax, [rsp+48] cmp rdi, rax je .L33 call operator delete(void*) .L33: mov rdi, rbx call _Unwind_Resume </pre>	<pre> movaps XMMWORD PTR [rsp+16], xmm0 jmp .L4 </pre>	
---	---	--

example_1_optimized() pros: avoided copy construction, avoided destructor call, smaller binary, faster code.

III. Motivating example #2

Let's change the above example to use a type, such as `std::variant`, with a non-trivial destructor:

```

#include <variant>
#include <string>
std::variant<int, std::string> produce();

static std::string get_string() {
    auto v = produce();
    if (auto* p = std::get_if<std::string>(&v)) {
        return *p;
    }
    return {};
}

int example_2() {
    return get_string().size();
}

```

Without user help, compilers are forced to copy construct `std::string` from `*p`.

Such code often appears in practice, where functions like `get_string()` are defined in header file of the library and `example_2()` like functions are written by some other developers in source file.

Experienced users may improve the above code to be more optimal:

```
#include <variant>
#include <string>
std::variant<int, std::string> produce();

static std::string get_string_move() {
    auto v = produce();
    if (auto* p = std::get_if<std::string>(&v)) {
        return std::move(*p);
    }
    return {};
}

int example_2_move() {
    return get_string_move().size();
}
```

Again, the above optimization **requires a highly experienced user** and, as in the first example, still does not produce a perfect result.

I assume that allowing copy elision for subobjects may allow compilers to transform `example_2()` into the code that is close to the following:

```
#include <variant>
#include <string>
std::variant<int, std::string> produce();

int example_2_optimized() {
    auto v = produce();
    if (auto* p = std::get_if<std::string>(&v)) {
        return p->size();
    }
    return 0;
}
```

Currently `[class.copy.elision]` prevents that optimization:

This elision of copy/move operations, called copy elision, is permitted in the following circumstances (which may be combined to eliminate multiple copies):

- in a return statement in a function with a class return type, when the expression is the name of a **non-volatile automatic object** (other than a function parameter or a variable introduced by the exception-declaration of a handler (18.3)) **with the same type** (ignoring cv-qualification) as the function return type, the copy/move operation can be omitted by constructing the automatic object directly into the function call's return object

Here's how the compiler optimized assemblies for the above 3 code snippets look like:

example_2() https://godbolt.org/g/BMC1QY	example_2_move() https://godbolt.org/g/WaV79e	example_2_optimized() https://godbolt.org/g/aT1Zmm
<pre>_erased_dtor<<skip>, 0ul>: rep ret _erased_dtor<<skip>, 1ul>: mov rdx, QWORD PTR [rdi] lea rax, [rdi+16] cmp rdx, rax je .L3 mov rdi, rdx jmp operator delete(void*) .L3: rep ret .LC0: .string "basic_string::_M_construct null not valid" .LC1: .string "basic_string::_M_create" example_2(): push r12 push rbp push rbx sub rsp, 80 lea rdi, [rsp+32] mov rbx, rsp call produce[abi:cxx11]() movzx eax, BYTE PTR [rsp+64] cmp al, 1 je .L35 lea rdx, [rbx+16] mov QWORD PTR [rsp+8], 0 mov BYTE PTR [rsp+16], 0 mov QWORD PTR [rsp], rdx .L13: cmp al, -1 je .L14 lea rdi, [rsp+32]</pre>	<pre>_erased_dtor<<skip>, 0ul>: rep ret _erased_dtor<<skip>, 1ul>: mov rdx, QWORD PTR [rdi] lea rax, [rdi+16] cmp rdx, rax je .L3 mov rdi, rdx jmp operator delete(void*) .L3: rep ret example_2_move(): push rbp push rbx sub rsp, 88 lea rdi, [rsp+32] mov rbx, rsp call produce[abi:cxx11]() movzx eax, BYTE PTR [rsp+64] lea rdx, [rbx+16] mov QWORD PTR [rsp], rdx cmp al, 1 je .L16 cmp al, -1 mov QWORD PTR [rsp+8], 0 mov BYTE PTR [rsp+16], 0 je .L10 .L9: lea rdi, [rsp+32] call [QWORD PTR _Variant_stg[0+rax*8]] .L10: mov rdi, QWORD PTR [rsp] add rbx, 16 mov rbp, QWORD PTR [rsp+8] cmp rdi, rbx</pre>	<pre>_erased_dtor<<skip>, 0ul>: rep ret _erased_dtor<<skip>, 1ul>: mov rdx, QWORD PTR [rdi] lea rax, [rdi+16] cmp rdx, rax je .L3 mov rdi, rdx jmp operator delete(void*) .L3: rep ret example_2_optimized(): push rbx xor ebx, ebx sub rsp, 48 mov rdi, rsp call produce[abi:cxx11]() movzx edx, BYTE PTR [rsp+32] cmp dl, -1 je .L5 cmp dl, 1 je .L12 .L7: mov rdi, rsp call [QWORD PTR _Variant_stg[0+rdx*8]] .L5: add rsp, 48 mov eax, ebx pop rbx ret .L12: mov ebx, DWORD PTR [rsp+8] jmp .L7 _Variant_stg: .quad _erased_dtor<<skip>, 0ul></pre>

<pre> call [QWORD PTR _Variant_stg[0+rax*8]] .L14: mov rdi, QWORD PTR [rsp] add rbx, 16 mov rbp, QWORD PTR [rsp+8] cmp rdi, rbx je .L5 call operator delete(void*) .L5: add rsp, 80 mov eax, ebp pop rbx pop rbp pop r12 ret .L35: mov r12, QWORD PTR [rsp+32] lea rax, [rbx+16] mov rbp, QWORD PTR [rsp+40] mov QWORD PTR [rsp], rax mov rax, r12 add rax, rbp je .L7 test r12, r12 je .L36 .L7: cmp rbp, 15 ja .L37 cmp rbp, 1 je .L38 test rbp, rbp jne .L19 .L34: mov rax, QWORD PTR [rsp] .L12: mov QWORD PTR [rsp+8], rbp mov BYTE PTR [rax+rbp], 0 movzx eax, BYTE PTR [rsp+64] jmp .L13 .L38: movzx eax, BYTE PTR [r12] mov BYTE PTR [rsp+16], al lea rax, [rbx+16] jmp .L12 .L37: test rbp, rbp js .L39 lea rdi, [rbp+1] call operator new(unsigned long) mov QWORD PTR [rsp], rax mov QWORD PTR [rsp+16], rbp .L10: mov rdx, rbp mov rsi, r12 mov rdi, rax call memcpy jmp .L34 .L19: lea rax, [rbx+16] jmp .L10 .L36: mov edi, OFFSET FLAT:.LC0 call std::__throw_logic_error movzx edx, BYTE PTR [rsp+64] mov rbx, rax cmp dl, -1 je .L18 lea rdi, [rsp+32] call [QWORD PTR _Variant_stg[0+rdx*8]] .L18: mov rdi, rbx call _Unwind_Resume .L39: mov edi, OFFSET FLAT:.LC1 call std::__throw_length_error _Variant_stg: .quad __erased_dtor<<skip>, 0ul> .quad __erased_dtor<<skip>, 1ul> </pre>	<pre> je .L5 call operator delete(void*) .L5: add rsp, 88 mov eax, ebp pop rbx pop rbp ret .L16: mov rdx, QWORD PTR [rsp+32] lea rcx, [rsp+48] cmp rdx, rcx je .L17 mov QWORD PTR [rsp], rdx mov rdx, QWORD PTR [rsp+48] mov QWORD PTR [rsp+16], rdx .L8: mov rdx, QWORD PTR [rsp+40] mov BYTE PTR [rsp+48], 0 mov QWORD PTR [rsp+40], 0 mov QWORD PTR [rsp+8], rdx lea rdx, [rsp+48] mov QWORD PTR [rsp+32], rdx .L9 .L17: movdqa xmm0, XMMWORD PTR [rsp+48] movaps XMMWORD PTR [rsp+16], xmm0 jmp .L8 _Variant_stg: .quad __erased_dtor<<skip>, 0ul> .quad __erased_dtor<<skip>, 1ul> </pre>	<pre> .quad __erased_dtor<<skip>, 1ul> </pre>
---	--	---

Pros: Avoided copy construction, avoided destructor call, smaller binary, faster code.

IV. Motivating example #3

This time we'll look at some code where compilers show the same results with copy elision and with `std::move`:

```

#include <utility>
#include <string>

```

```
#include <memory>

struct some_type {
    short i;
};

std::pair<int, std::shared_ptr<some_type>> produce();

static std::shared_ptr<some_type> return_second() {
    auto v = produce();
    return v.second;
}

int example_3() {
    return !!return_second();
}
```

With std::move:

```
#include <utility>
#include <string>
#include <memory>

struct some_type {
    short i;
};

std::pair<int, std::shared_ptr<some_type>> produce();

static std::shared_ptr<some_type> return_second_move() {
    auto v = produce();
    return std::move(v.second);
}

int example_3_move() {
    return !!return_second_move();
}
```

Code that emulates copy elision:

```
#include <utility>
#include <string>
#include <memory>

struct some_type {
    short i;
};

std::pair<int, std::shared_ptr<some_type>> produce();

int example_3_optimized() {
    return !!produce().second;
}
```

example_3() https://godbolt.org/g/LcTD1y	example_3_move() https://godbolt.org/g/HMqKAr	example_3_optimized() https://godbolt.org/g/5t3t3t
<pre><skip>::~_M_dispose(): rep ret <skip>_M_destroy(): mov rax, QWORD PTR [rdi] jmp [QWORD PTR [rax+8]] example_3(): push r13 push r12 push rbp push rbx sub rsp, 56 lea rdi, [rsp+16] call produce() mov rbx, QWORD PTR [rsp+32] mov rax, QWORD PTR [rsp+24] test rbx, rbx je .L34 test rax, rax lea rbp, [rbx+8] mov r12d, OFFSET FLAT:__gthrw__pthread_key_create setne al test r12, r12 mov rcx, rbp movzx eax, al je .L7 lock add DWORD PTR [rbp+0], 1 mov r13, QWORD PTR [rsp+32] test r13, r13 lea rcx, [r13+8] je .L23 test r12, r12 je .L10</pre>	<pre><skip>::~_M_dispose(): rep ret <skip>_M_destroy(): mov rax, QWORD PTR [rdi] jmp [QWORD PTR [rax+8]] example_3_move(): push rbp push rbx sub rsp, 56 lea rdi, [rsp+16] call produce() xor eax, eax cmp QWORD PTR [rsp+24], 0 mov rbx, QWORD PTR [rsp+32] setne al test rbx, rbx je .L4 mov ebp, OFFSET FLAT:__gthrw__pthread_key_create test rbp, rbp je .L7 mov edx, -1 lock xadd DWORD PTR [rbx+8], edx cmp edx, 1 je .L18 .L4: add rsp, 56 pop rbx pop rbp ret .L7: mov edx, DWORD PTR [rbx+8] lea ecx, [rdx-1]</pre>	<pre><skip>::~_M_dispose(): rep ret <skip>_M_destroy(): mov rax, QWORD PTR [rdi] jmp [QWORD PTR [rax+8]] example_3_optimized(): push rbp push rbx sub rsp, 56 lea rdi, [rsp+16] call produce() xor eax, eax cmp QWORD PTR [rsp+24], 0 mov rbx, QWORD PTR [rsp+32] setne al test rbx, rbx je .L4 mov ebp, OFFSET FLAT:__gthrw__pthread_key_create test rbp, rbp je .L7 mov edx, -1 lock xadd DWORD PTR [rbx+8], edx cmp edx, 1 je .L18 .L4: add rsp, 56 pop rbx pop rbp ret .L7: mov edx, DWORD PTR [rbx+8] lea ecx, [rdx-1]</pre>

```

.L37:
mov     edx, -1
lock xadd    DWORD PTR [rcx], edx
cmp     edx, 1
je      .L35
.L23:
test    r12, r12
je      .L17
mov     edx, -1
lock xadd    DWORD PTR [rbp+0], edx
cmp     edx, 1
je      .L36
.L4:
add     rsp, 56
pop     rbx
pop     rbp
pop     r12
pop     r13
ret
.L7:
add     DWORD PTR [rbx+8], 1
test    r12, r12
mov     r13, rbx
jne     .L37
.L10:
mov     edx, DWORD PTR [r13+8]
lea     ecx, [rdx-1]
cmp     edx, 1
mov     DWORD PTR [r13+8], ecx
jne     .L23
.L35:
mov     rdx, QWORD PTR [r13+0]
mov     rdx, QWORD PTR [rdx+16]
cmp     rdx, OFFSET FLAT:<skip>::_M_dispose()
jne     .L38
.L13:
test    r12, r12
je      .L14
mov     edx, -1
lock xadd    DWORD PTR [r13+12], edx
.L15:
cmp     edx, 1
jne     .L23
mov     rdx, QWORD PTR [r13+0]
mov     DWORD PTR [rsp+12], eax
mov     rdi, r13
mov     rcx, QWORD PTR [rdx+24]
cmp     rcx, OFFSET FLAT:<skip>::_M_destroy()
jne     .L16
call    [QWORD PTR [rdx+8]]
mov     eax, DWORD PTR [rsp+12]
jmp     .L23
.L34:
test    rax, rax
setne   al
add     rsp, 56
pop     rbx
movzx   eax, al
pop     rbp
pop     r12
pop     r13
ret
.L17:
mov     edx, DWORD PTR [rbx+8]
lea     ecx, [rdx-1]
cmp     edx, 1
mov     DWORD PTR [rbx+8], ecx
jne     .L4
.L36:
mov     rdx, QWORD PTR [rbx]
mov     rdx, QWORD PTR [rdx+16]
cmp     rdx, OFFSET FLAT:<skip>::_M_dispose()
jne     .L39
.L19:
test    r12, r12
je      .L20
mov     edx, -1
lock xadd    DWORD PTR [rbx+12], edx
.L21:
cmp     edx, 1
jne     .L4
mov     rdx, QWORD PTR [rbx]
mov     DWORD PTR [rsp+12], eax
mov     rdi, rbx
mov     rcx, QWORD PTR [rdx+24]
cmp     rcx, OFFSET FLAT:<skip>::_M_destroy()
jne     .L22
call    [QWORD PTR [rdx+8]]
mov     eax, DWORD PTR [rsp+12]
jmp     .L4
.L14:
mov     edx, DWORD PTR [r13+12]

```

```

cmp     edx, 1
mov     DWORD PTR [rbx+8], ecx
jne     .L4
.L18:
mov     rdx, QWORD PTR [rbx]
mov     rdx, QWORD PTR [rdx+16]
cmp     rdx, OFFSET FLAT:<skip>::_M_dispose()
jne     .L19
.L10:
test    rbp, rbp
je      .L11
mov     edx, -1
lock xadd    DWORD PTR [rbx+12], edx
.L12:
cmp     edx, 1
jne     .L4
mov     rdx, QWORD PTR [rbx]
mov     DWORD PTR [rsp+12], eax
mov     rdi, rbx
mov     rcx, QWORD PTR [rdx+24]
cmp     rcx, OFFSET FLAT:<skip>::_M_destroy()
jne     .L13
call    [QWORD PTR [rdx+8]]
mov     eax, DWORD PTR [rsp+12]
jmp     .L4
.L11:
mov     edx, DWORD PTR [rbx+12]
lea     ecx, [rdx-1]
mov     DWORD PTR [rbx+12], ecx
jmp     .L12
.L19:
mov     DWORD PTR [rsp+12], eax
mov     rdi, rbx
call    rdx
mov     eax, DWORD PTR [rsp+12]
jmp     .L10
.L13:
call    rcx
mov     eax, DWORD PTR [rsp+12]
jmp     .L4

```

```

cmp     edx, 1
mov     DWORD PTR [rbx+8], ecx
jne     .L4
.L18:
mov     rdx, QWORD PTR [rbx]
mov     rdx, QWORD PTR [rdx+16]
cmp     rdx, OFFSET FLAT:<skip>::_M_dispose()
jne     .L19
.L10:
test    rbp, rbp
je      .L11
mov     edx, -1
lock xadd    DWORD PTR [rbx+12], edx
.L12:
cmp     edx, 1
jne     .L4
mov     rdx, QWORD PTR [rbx]
mov     DWORD PTR [rsp+12], eax
mov     rdi, rbx
mov     rcx, QWORD PTR [rdx+24]
cmp     rcx, OFFSET FLAT:<skip>::_M_destroy()
jne     .L13
call    [QWORD PTR [rdx+8]]
mov     eax, DWORD PTR [rsp+12]
jmp     .L4
.L11:
mov     edx, DWORD PTR [rbx+12]
lea     ecx, [rdx-1]
mov     DWORD PTR [rbx+12], ecx
jmp     .L12
.L19:
mov     DWORD PTR [rsp+12], eax
mov     rdi, rbx
call    rdx
mov     eax, DWORD PTR [rsp+12]
jmp     .L10
.L13:
call    rcx
mov     eax, DWORD PTR [rsp+12]
jmp     .L4

```

<pre> lea ecx, [rdx-1] mov DWORD PTR [r13+12], ecx jmp .L15 .L20: mov edx, DWORD PTR [rbx+12] lea ecx, [rdx-1] mov DWORD PTR [rbx+12], ecx jmp .L21 .L39: mov DWORD PTR [rsp+12], eax mov rdi, rbx call rdx mov eax, DWORD PTR [rsp+12] jmp .L19 .L38: mov DWORD PTR [rsp+12], eax mov rdi, r13 call rdx mov eax, DWORD PTR [rsp+12] jmp .L13 .L16: call rcx mov eax, DWORD PTR [rsp+12] jmp .L23 .L22: call rcx mov eax, DWORD PTR [rsp+12] jmp .L4 </pre>		
--	--	--

Pros: Avoided copy construction, avoided destructor call, smaller binary, faster code.

Even when there's no benefit to be had over using `std::move`, it is better to have that benefit automatically without requiring user intervention.

V. Proposed wording 1: A simple solution for relaxing [class.copy.elision]

Modify the [class.copy.elision] paragraph 1 to allow copy elision for returning subobjects of the objects with defaulted destructor:

```

This elision of copy/move operations, called copy elision, is permitted in the following
circumstances (which may be combined to eliminate multiple copies):

<...>

- in a return statement in a function with a class return type, when
    • the expression is a non-volatile subobject of the non-volatile local object
    • and the non-volatile local object has defaulted destructor
    • and the type of subobject is same (ignoring cv-qualification) as the function return type
    • and the subobject is not accessed between a copy/move construction of it and its destruction.

the copy/move operation can be omitted by inlining the function call, constructing the automatic
object directly in the caller, calling the destructors for non-returned subobjects and treating the
target as another way to refer to the subobject.

<...>

```

This will allow optimizing `std::pair`, `std::tuple` and all the aggregate initializable types.

VI. Proposed wording 2: A generic solution for relaxing [class.copy.elision]

Modify the [class.copy.elision] paragraph 1 to allow copy elision for returning subobjects of the objects (dropped the "defaulted destructor" requirement):

```

This elision of copy/move operations, called copy elision, is permitted in the following
circumstances (which may be combined to eliminate multiple copies):

<...>

- in a return statement in a function with a class return type, when
    • the expression is a non-volatile subobject of the non-volatile local object
    • and the type of subobject is same (ignoring cv-qualification) as the function return type
    • and the subobject is not accessed between a copy/move construction of it and its destruction.

the copy/move operation can be omitted by inlining the function call, constructing the automatic
object directly in the caller, calling the destructors for non-returned subobjects and treating the
target as another way to refer to the subobject.

<...>

```

This will allow optimizing `std::pair`, `std::tuple`, all the aggregate initializable types AND probably `std::variant`, `std::optional`, many user provided classes.

VII. Proposed wording 3: An ultimate solution for relaxing [class.copy.elision]

Use the "Ultimate copy elision" wording from our companion paper P0889R0.

VIII. Minor points

Automatically applying `std::move` to returned subobjects may be a good idea, but for a separate proposal. As was shown in the first two examples, copy elision could be profitable even with moving out the subobject.

It would be wrong to guarantee/require the above optimizations as they are highly dependent on inlining and aliasing reasoning of the compiler.

Copy elision of subobjects could be useful in many cases. The paper tries to allow more optimizations rather than forcing some particular optimizations. Thus, compiler vendors may find alternatives to the above optimizations.

IX. Acknowledgements

Many thanks to Walter E. Brown for fixing numerous issues in draft versions of this paper.

Many thanks to Eyal Rozenberg for numerous useful comments.

Many thanks to Vyacheslav Napadovsky for helping with the assembly and the optimizations.

Thanks also to Marc Glisse for pointing me to the forbidden copy elision cases, and to Thomas Köppe, Peter Dimov, Richard Smith, Gabriel Dos Reis, Anton Bikineev for participating in early discussions and thus helping me to assemble my thoughts.

X. Feature-testing macro

For the purposes of SG10, we recommend the feature-testing macro name `__cpp_subobject_copy_elision`.